CASE STUDY: DICTIONARY APPLICATION USING HASH TABLES

1. Introduction

A dictionary is an application that stores words along with their meanings and allows users to search for words quickly. Examples include online dictionaries, language-learning applications, and spell-checking systems.

A hash table is a data structure that stores information using a hash function. The hash function converts a key, such as a word, into an index in an array where the corresponding meaning can be stored.

In this case study, a dictionary application is implemented using a hash table with **separate chaining** to resolve collisions.

2. Problem Statement

A dictionary application must store a large collection of words and their meanings. Users should be able to add new words, search for existing words, update meanings, and delete words efficiently.

The challenge is that different words may produce the same hash-table index. This situation is called a collision and must be handled correctly.

The objective is to design a dictionary application that uses hashing and collision resolution to provide efficient storage and retrieval of words and meanings.

3. Objectives

- To understand hash tables and hash functions.
- To implement a dictionary using hashing.
- To handle collisions using separate chaining.
- To perform insertion, searching, updating, and deletion.
- To analyze the time and space complexity of hash-table operations.

4. Proposed System

The proposed system uses a hash table consisting of an array of buckets. Each bucket contains a list of key-value pairs.

- **Key:** The dictionary word.
- **Value:** The meaning of the word.
- **Hash function:** Converts a word into a bucket index.
- **Collision resolution:** Separate chaining stores multiple entries in the same bucket when their hash indices match.

The index is calculated as:

[
h(k)=\text{hash}(k)\bmod m
]

Where:

- (k) is the word.
- (m) is the number of buckets.
- (h(k)) is the resulting bucket index.

For a simple demonstration, the implementation below uses a custom hash function based on the characters of each word.

5. Example Dataset

Word	Meanin
Apple	An edible fruit
Book	A collection of written pages

Computer	An electronic device for processing data
Dictionary	A reference containing words and meanings
Education	The process of learning
Hashing	A technique for mapping keys to indices

When a user enters a word, the hash function calculates the appropriate bucket. The application then searches that bucket for the requested word.

6. Collision Resolution Technique

A collision occurs when two different words map to the same index.

For example, suppose the hash function maps both `cat` and `act` to bucket 2. The hash table cannot store only one entry in that bucket without losing information.

Separate chaining solves this problem by storing multiple key-value pairs in a list at the same bucket.

```
Hash Table

Bucket 0  -> [apple, meaning]
Bucket 1  -> [book, meaning]
Bucket 2  -> [cat, meaning] -> [act, meaning]
Bucket 3  -> [computer, meaning]
Bucket 4  -> Empty
```

The example is illustrative: actual bucket positions depend on the hash function and table size.

7. Algorithm

A. Insertion

1. Accept a word and its meaning.
2. Calculate the word's hash index.
3. Access the corresponding bucket.
4. If the word already exists, update its meaning.
5. Otherwise, add the word and meaning to the bucket.

B. Searching

1. Accept the word to search for.
2. Calculate its hash index.
3. Examine the entries in the corresponding bucket.
4. If the word is found, display its meaning.
5. Otherwise, display "Word not found."

C. Deletion

1. Accept the word to delete.
2. Calculate its hash index.
3. Search the corresponding bucket.
4. Remove the matching entry if it exists.
5. Otherwise, display "Word not found."

8. Python Implementation

```
class DictionaryHashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    # Custom hash function
    def hash_function(self, word):
```

```
        word = word.lower()
        total = 0

        for char in word:
            total = total + ord(char)

        return total % self.size

    # Insert or update a word
    def insert(self, word, meaning):
        index = self.hash_function(word)
        bucket = self.table[index]

        for i, (existing_word, _) in enumerate(bucket):
            if existing_word.lower() == word.lower():
                bucket[i] = (word, meaning)
                print("Meaning updated successfully.")
                return

        bucket.append((word, meaning))
        print("Word added successfully.")

    # Search for a word
    def search(self, word):
        index = self.hash_function(word)

        for existing_word, meaning in self.table[index]:
            if existing_word.lower() == word.lower():
                return meaning

        return None

    # Delete a word
    def delete(self, word):
        index = self.hash_function(word)
        bucket = self.table[index]

        for i, (existing_word, _) in enumerate(bucket):
            if existing_word.lower() == word.lower():
                bucket.pop(i)
                print("Word deleted successfully.")
                return

        print("Word not found.")

    # Display all words
    def display(self):
        print("\nDictionary Contents:")

        for index, bucket in enumerate(self.table):
            print(f"Bucket {index}: {bucket}")


# Main program
dictionary = DictionaryHashTable()

dictionary.insert("Apple", "An edible fruit")
dictionary.insert("Book", "A collection of written pages")
dictionary.insert("Cat", "A small domesticated animal")
dictionary.insert("Act", "To do something")
```

```
print("\nSearching for Apple:")
print(dictionary.search("Apple"))

print("\nSearching for Book:")
print(dictionary.search("Book"))

print("\nSearching for Mango:")
meaning = dictionary.search("Mango")
print(meaning if meaning else "Word not found.")

dictionary.insert("Apple", "A sweet edible fruit")

dictionary.delete("Cat")

dictionary.display()
```

9. Sample Output

```
Word added successfully.
Word added successfully.
Word added successfully.
Word added successfully.

Searching for Apple:
An edible fruit

Searching for Book:
A collection of written pages

Searching for Mango:
Word not found.

Meaning updated successfully.
Word deleted successfully.

Dictionary Contents:
Bucket 0: ...
Bucket 1: ...
...
```

The bucket contents depend on the selected hash-table size and the hash values of the words. The program prints the actual bucket contents when executed.

10. Working of the System

The application performs the following steps:

1. The user enters a word and its meaning.
2. The hash function calculates the bucket index.
3. The word and meaning are stored in the corresponding bucket.
4. If another word maps to the same bucket, separate chaining stores it alongside the existing entry.
5. When a word is searched, only its corresponding bucket needs to be examined.
6. The application returns the meaning if the word exists.

This reduces unnecessary comparisons compared with searching through every dictionary entry.

11. Time and Space Complexity

Let (n) be the number of words and (m) the number of buckets.

Operation	Average Case	Worst Case

Insertion	O(1)	O(n)
Searching	O(1)	O(n)
Deletion	O(1)	O(n)
Display all entries	O(n + m)	O(n + m)

The average-case performance assumes a suitable hash function and a reasonably distributed set of words. If all words map to one bucket, searching, insertion, and deletion can take O(n) time.

The space complexity is O(n + m), accounting for stored entries and the bucket array.

12. Advantages

- Fast average-case word lookup.
- Efficient insertion and deletion.
- Collision handling prevents entries from being lost.
- Dynamic storage of words and meanings.
- Suitable for dictionaries with large numbers of entries.

13. Limitations

- Performance can degrade when many words collide.
- The quality of the hash function affects performance.
- Hash tables do not naturally maintain words in alphabetical order.
- The simple implementation stores data in memory and does not preserve it after the program exits.
- A production dictionary may require persistent storage, input validation, Unicode normalization, and more advanced language-processing features.

14. Applications

- English and multilingual dictionary applications.
- Spell-checking software.
- Autocomplete and word lookup systems.
- Translation applications.
- Vocabulary-learning tools.
- Symbol tables in compilers.
- Key-value storage systems.

15. Conclusion

This case study demonstrates how a dictionary application can be implemented using a hash table. Words act as keys, while their meanings act as values. A hash function maps each word to a bucket, and separate chaining resolves collisions by allowing multiple entries in the same bucket.

Hash tables provide average-case O(1) insertion, searching, and deletion under suitable conditions. Therefore, they are an effective data structure for dictionary applications and other systems that require fast key-based retrieval.